DENIAL OF WALLET

Architectural defense for pay-as-you-go infrastructure — WebSockets, egress bandwidth, and per-request billing at the edge.

a.k.a. **EDoS** — Economic Denial of Sustainability // the outage is optional; the invoice is the payload.

THE LAW

metered resource + self-serve limit raise + unauthenticated endpoint = Denial-of-Wallet primitive

The billing meter is the attack surface. If cost scales with traffic an attacker controls, and nothing authenticates that traffic, **the bill is a weapon aimed at the account owner** — no exploit required.

§1 TWO LEVERS — KEEP THEM DISTINCT

PRESSURE VS. BILL

LEVER 1 — THE CAP

Denial of Service

→ the pressure

Saturate the concurrent-connection ceiling. Real users are refused (5xx). This does not cost the attacker anything meaningful; it costs the owner customers.

LEVER 2 — THE METER

Denial of Wallet

→ the bill

Pay-as-you-go on connection-time and bandwidth. No cap is hit; the number that moves is the monthly invoice. Silent, uncapped, attacker-driven.

> **The conversion IS the trap:** raising the cap to stop the 5xx turns a Denial of Service into a Denial of Wallet. You buy your way out of the outage and straight into the bill.

§2 THE ECONOMICS — BAIT VS. WEAPON

WORKED ON A REAL EDGE-WS PRICE LIST

```
# CONNECTION-TIME  — cheap, and bounded by the cap
  rate            $0.235  per 1,000,000 connection-minutes
  worst case      25,000 conns (max self-serve)  ×  30 days
                  25,000 × 43,200 min  =  1,080,000,000 conn-min
  monthly ceiling ≈ $254  →  a quarter-thousand dollars scares no one

# BANDWIDTH  — metered like CDN traffic, effectively unbounded
  the attacker does not idle sockets — it pushes payload through them
  cost scales with data volume, not with connection count
```

Connection-time is the ticket in. **Bandwidth is the knife.** A defender who only watches the connection cap is guarding the door while the window is open.

RULE ▸ Meter and alarm connection-count, connection-time, and bandwidth as three independent signals. The dangerous one is bandwidth.

§3 THE TRAP — THE PANIC LOOP

1. Attacker saturates the connection cap.
2. Real clients start receiving 5xx.
3. Owner raises the cap to stop the errors — *"better to pay than to drop customers."*
4. Bigger bill **and** a larger abuse surface than before.

↺ Loop repeats — each round is more expensive.

The loop only closes if the limit-raise is **automatic**. Remove the automation and the trap has no teeth: a ceiling reached becomes an incident, not a purchase.

§4 THE DEFENSE — ARCHITECTURAL, NOT TACTICAL

CLOSE IT AT THE DESIGN LAYER

1 Authenticate before the upgrade

The anonymous WebSocket endpoint **is** the hole. Require a signed token bound to a real session **before** the HTTP→WS upgrade completes. Now holding N connections costs the attacker N real accounts — the whole economy inverts.

2 Mandatory heartbeat + aggressive idle timeout

Zombie connections are the fuel of connection-time. Force a ping/pong; drop anything silent fast. **Starve the meter of idle sockets.**

3 Hard-cap and alarm on bandwidth — not just connections

Bandwidth is the weapon, so watch the weapon. A per-connection and per-tenant byte-rate spike is an **incident**, not a metric. Cap it before it caps your budget.

4 Per-IP / per-ASN limits at the edge

A single origin must not be able to hold thousands of sockets. Bound concurrency by source before traffic ever reaches the meter.

5 A cap-hit is an alert, never a button

Never auto-raise a **paid** limit. A reached ceiling is a signal to investigate, page a human, and decide — not a checkout to approve on reflex.

§5 SAME CLASS, EVERYWHERE

NOT NEW, NOT VENDOR-SPECIFIC

Any metered resource with a self-serve ceiling and an unauthenticated path is the same primitive. The names change; the shape does not.

- **Object storage.** In 2024 a widely-reported case showed bucket owners billed for unauthorized (403) requests they never served or consented to; the provider then changed billing so unauthorized requests are no longer charged to the owner. The fix was to stop metering what the owner did not authorize.
- **Serverless, egress, per-request APIs.** Identical shape — cost scales with attacker-controlled volume, and the owner pays for work they never asked for.

The lesson is always the same: **the mitigation is architectural.** It never runs through an unauthenticated, anonymously-metered endpoint exposed to the open internet.

THE DECISION RULE — PIN THIS LINE

A reached limit is an incident, not an auto-approval. Pre-commit your maximum spend. Fail toward a bounded bill — never an unbounded one.